

1. Por onde começar

Você não precisa colocar seus programas em src. Para escrever um programa que roda no processador Mancha, crie um arquivo `.asm` em `simulador_completo/exemplos/`. Para acrescentar um dispositivo ao simulador, altere o código C em `simulador_completo/src/`. São dois tipos diferentes de trabalho.

O que o trecho do professor pede

Trecho recebido	Como se orientar
Compilar e entender exemplos <code>.asm</code>	Etapa de familiarização: montar, executar e observar registradores e console.
Imprimir instruções e segundos	Exercício sugerido de medição. O código atual exige cuidados e possivelmente novas portas de leitura.
Ler 10 ou n valores do novo dispositivo	Sugestão que pressupõe a existência de um dispositivo. O trecho não informa seu contrato.
O que deve ser entregue: gerar um número aleatório	É a entrega explicitamente destacada. O trecho não esclarece sozinho se basta um programa ou se o gerador deve ser um novo dispositivo.

Interpretação provável: a menção a “esse novo dispositivo” sugere implementar um gerador no simulador e escrever Assembly para acessá-lo. Isso é uma interpretação do trecho, não uma exigência adicional confirmada. Verifique no enunciado completo se há porta, faixa dos números ou linguagem obrigatória para a entrega.

Os dois caminhos do projeto

Programa Assembly: `.asm -> montador -> .mob -> simulador`

Programa C: `.c -> mcc -> .asm -> montador -> .mob`

O processador executa instruções de máquina carregadas do `.mob`. O `mcc` é um compilador auxiliar; ele não faz parte da CPU. Nos exercícios que dizem “em linguagem de montagem”, escreva o programa em Assembly.

Este guia explica a implementação local encontrada nesta pasta. Os trechos marcados como proposta não estão implementados. Nenhum código do simulador foi alterado para produzir este PDF.

2. Onde criar cada coisa

O que você quer fazer	Local indicado
Programa Assembly do T1	simulador_completo/exemplos/t1_aleatorio.asm (nome sugerido)
Sub-rotina Assembly	Um rótulo no próprio .asm, chamado com call e finalizado com ret.
Novo dispositivo de entrada/saída	simulador_completo/src/dispositivos.c e dispositivos.h
Mudar execução ou contagem da CPU	simulador_completo/src/cpu.c e cpu.h
Mudar comandos e controle da aplicação	simulador_completo/src/main.c
Mudar a interface da console	simulador_completo/src/tela.c
Programa C compilado para Mancha	compilador_c/exemplos/meuprograma.c
Alterar o compilador C	compilador_c/src/ (lexer, parser, tipos etc.)

Quando preciso criar um novo arquivo .c em src?

Somente se decidir separar a implementação do simulador em um módulo. Para um dispositivo pequeno, você pode começar dentro de dispositivos.c. Uma função auxiliar usada apenas nesse arquivo pode ser **static**; uma função chamada por outro módulo precisa de declaração no cabeçalho correspondente.

Se criar, por exemplo, **src/aleatorio.c** e **src/aleatorio.h**, inclua o cabeçalho onde for necessário e adicione **\$(SRC)/aleatorio.c** à variável **SIM_SRC** do Makefile do simulador. A lista de fontes C é explícita: apenas criar o arquivo não o inclui na compilação.

Quando basta criar um .asm?

Quando você usa as instruções e dispositivos já existentes. O Makefile procura automaticamente **exemplos/*.asm** e gera os .mob. Por isso, um novo .asm nessa pasta passa a ser montado com **make -C simulador_completo**.

Mantenha o Assembly escrito à mão em simulador_completo/exemplos/. A pasta compilador_c/exemplos/ contém Assembly gerado do C; o comando make clean do compilador remove esses .asm.

3. Recursos que já existem

Instruções úteis para o T1

Grupo	Instruções e uso
Dados	ld / ldb: carregar; st / stb: guardar; ldq: carregar constante pequena.
Contas e bits	add, sub, mul, div, cmp; and, or, xor; shl e shr.
Decisões e repetição	jmp; jmpc com condições como z, nz, lt, ge; cmp prepara a comparação.
Sub-rotinas e pilha	call, ret, push e pop. Combine quais registradores a rotina preserva.
Entrada e saída	in / inb leem portas; out / outb escrevem portas.
Controle	halt encerra; di / ei controlam interrupções; trap / rete tratam exceções.

Exemplo: **ld r0, 10** carrega o número 10; **ld r0, (valor)** lê a memória no endereço do rótulo valor; **in r0, (1)** lê a porta 1. A instrução define se o endereço é de memória ou de dispositivo.

Portas implementadas nesta versão

Endereço hexadecimal	Recurso
0x0001	Console: escrever envia um caractere; ler retira um caractere recebido.
0x0002	Estado: bit 0 = pronta para saída; bit 1 = existe entrada.
0x0010 a 0x0015	Disco: face, trilha, setor, operação e endereço de memória.
0x0020 / 0x0021	Byte alto / baixo do contador do relógio simulado.
0x0022 / 0x0023	Byte alto / baixo do limite do relógio.
0x0030	Máscara do controlador de interrupções.

Não existe porta aleatória no código atual. Uma leitura de porta desconhecida retorna zero. As portas retornam 8 bits mesmo com in; para um valor maior, é preciso definir um protocolo com vários bytes. Os exemplos inicializam o modo supervisor, que permite as operações usadas aqui.

O relógio só incrementa quando seu limite é diferente de zero; ao atingir o limite, zera o contador e sinaliza uma interrupção. Ele não fornece segundos reais.

4. Seu primeiro arquivo Assembly

Crie **simulador_completo/exemplos/t1_inicio.asm** com este exemplo didático. Ele mostra como criar uma “função” em Assembly: um rótulo chamado por call. A saída esperada é **7** seguido de uma quebra de linha.

```
.equ pilha = 0x3F0
.org 0
.dw main, pilha, 0, 0

.org 0x80
main:
    ldq r0, 7
    call imprime_digito
    ldq r0, 10
    out r0, (1)
    halt

; Entrada: r0 entre 0 e 9
; Altera: r0 e flags
imprime_digito:
    add r0, 48
    out r0, (1)
    ret
```

.equ define uma constante. **.org** escolhe o endereço de montagem. O **.dw** inicial fornece IP, SP, CS e DS para o quadro de inicialização. **main** é um rótulo; aqui o quadro inicial aponta para ele. A pilha permite que call guarde o retorno e ret volte à instrução seguinte.

Montar e executar, a partir da pasta principal

```
make -C simulador_completo
./simulador_completo/bin/simulador \
    simulador_completo/exemplos/t1_inicio.mob
```

O Makefile monta o novo exemplo automaticamente. O código acima foi preparado conforme a sintaxe dos exemplos locais; sua execução no WSL não foi realizada durante a criação deste guia.

Por que não usar printf?

A console usa ncurses para desenhar a tela. No programa Assembly, envie caracteres para a porta 1 com out. No código C do simulador, não acrescente printf para exibir os resultados do programa durante essa interface. Use o caminho de saída da console.

Número não é caractere: out com o valor 7 não escreve o texto “7”. Para um dígito, some 48 (ASCII de “0”). Para 42, envie os caracteres “4” e “2”. A rotina acima só aceita 0 a 9.

5. Como acrescentar o gerador

Se o T1 exigir um **novo dispositivo aleatório**, uma organização simples é: implementar a geração em C dentro do simulador e usar `in` no programa Assembly para obter cada valor. Você não precisa inventar uma nova instrução de CPU.

Contrato sugerido - ainda não implementado

Reserve uma porta livre, por exemplo **0x0040**, após conferir o enunciado completo. Cada leitura devolve um novo valor de **0 a 255**. A porta e a faixa são sugestões compatíveis com a interface de 8 bits atual, não especificações do professor.

Etapa	Mudança no simulador
1. Nomear a porta	Em <code>dispositivos.h</code> , definir <code>PORTA_ALEATORIO</code> com o endereço escolhido.
2. Guardar estado	Em <code>struct dispositivos</code> , manter o estado do gerador, se a estratégia escolhida precisar dele.
3. Inicializar	Em <code>disp_cria</code> , inicializar o estado uma vez por sessão; evitar reinicializar a semente a cada leitura.
4. Responder à leitura	No switch de <code>disp_le_byte</code> , adicionar o case da porta: atualizar o gerador e retornar o byte.
5. Definir reinício	Decidir se <code>R</code> preserva ou reinicia o gerador. Hoje <code>cpu_reinicia</code> não recria os dispositivos.
6. Recompilar	Executar <code>make -C simulador_completo</code> e testar um <code>.asm</code> consumidor.

Fluxo de uma leitura

```
Assembly: in r0, (0x0040)
-> CPU interpreta a instrucao in
-> disp_le_byte recebe a porta
-> dispositivo gera e retorna um byte
-> r0 recebe o valor
```

Aleatório e pseudoaleatório: um gerador determinístico repete a sequência quando começa com a mesma semente. Para variar entre sessões, a inicialização precisa variar. Uma semente baseada apenas em segundos pode repetir em execuções rápidas. Números iguais em duas leituras também são possíveis: aleatoriedade não garante unicidade.

O trecho do professor não define se “cada vez que é executado” significa cada nova sessão ou cada leitura do dispositivo. Documente sua escolha e confira esse detalhe no enunciado completo. O objetivo aqui é orientar sua implementação, sem apresentar a proposta como recurso já pronto.

6. Ler e imprimir 10 valores

Depois de implementar e testar a porta, escreva **t1_aleatorio.asm**. Comece lendo um valor; só depois amplie para dez e, se desejar, ordene. Esse desenvolvimento em etapas ajuda a localizar os erros.

Estrutura do algoritmo em Assembly

```
; Pseudocódigo: não é um programa pronto para montar
contador = 10
enquanto contador > 0:
    valor = ler_porta_aleatoria()
    imprimir_decimal(valor)
    imprimir_quebra_de_linha()
    contador = contador - 1
parar
```

No .asm real, use um registrador ou uma posição de memória para o contador; `cmp` e `jmpc` para a decisão; `in` para a leitura; `call` para chamar a rotina de impressão; `out` para enviar os caracteres; `halt` para finalizar.

Como imprimir de 0 a 255

Uma estratégia simples evita depender de uma função pronta: copie o valor para um resto, subtraia 100 repetidamente para contar centenas e depois 10 para contar dezenas. O resto final é a unidade. Converta cada dígito em ASCII somando 48 e envie-o à porta 1.

Imprima a centena somente quando ela existir; imprima a dezena quando houver centena ou dezena não nula; imprima sempre a unidade. Assim, 0 sai como "0", 7 como "7" e 105 como "105". Outra opção inicial é imprimir sempre três posições, como "007".

Defina o contrato da rotina: por exemplo, recebe o valor em `r0` e preserva o registrador usado pelo laço. Use `push/pop` para preservar registradores quando necessário. Não suponha que `call` salva todos eles automaticamente.

Se quiser ordenar

Primeiro armazene todos os valores num vetor de memória. Depois ordene e imprima. Com dez valores, uma ordenação simples por comparação e troca é suficiente para estudar. Se guardar palavras de 16 bits, cada elemento ocupa dois bytes; se guardar bytes, use `ldb/stb` e avance um byte.

Verificação prática

Confira: exatamente dez valores; todos na faixa combinada; números com um, dois e três dígitos legíveis; término com `halt`; novas execuções funcionando. Na versão ordenada, verifique ordem não decrescente e preservação de valores repetidos.

Antes de validar a aleatoriedade, teste a impressão com valores conhecidos como 0, 7, 10, 99, 100 e 255. Isso separa falhas de formatação de falhas no gerador.

7. Instruções e segundos

Essa sugestão do professor envolve uma diferença importante entre o tempo do computador que executa o simulador e o tempo modelado pelo simulador.

O que já está disponível no C do simulador

Função	O que ela oferece
<code>cpu_executa_1(cpu)</code>	Avança uma unidade de simulação: uma instrução ou atendimento de interrupção.
<code>cpu_num_instrucoes(cpu)</code>	Consulta o contador interno. Ele também aumenta no atendimento de interrupções e em passos que terminam com falha.
<code>disp_tick(d)</code>	Avança dispositivos. O relógio avança se seu limite estiver configurado.
<code>disp_relogio_contador(d)</code>	Consulta o contador de 16 bits do relógio simulado.
<code>disp_le_byte / disp_escreve_byte</code>	São a ponte entre instruções in/out e os dispositivos.

Essas funções são chamadas no **C do simulador**. Um `.asm` não consegue chamar `cpu_num_instrucoes` por nome. Para consultá-lo pelo programa simulado, você precisa expor o dado, por exemplo, através de novas portas.

Por que o relógio existente não mede segundos?

Ele avança por passos e volta a zero ao alcançar o limite. A velocidade da interface altera quantos passos ocorrem em um segundo real. Portanto, não transforme o contador em segundos usando uma taxa fixa sem que isso tenha sido definido e implementado no modelo.

Plano para a extensão de medição

1. Defina a métrica: contar passos como hoje ou contar apenas instruções segundo um critério explícito. Defina também se segundos incluem pausas e espera por entrada.
2. Para tempo real decorrido, registre um instante inicial e use um relógio monotônico no código hospedeiro. Se quiser tempo somente em execução, acumule apenas intervalos em que a simulação estiver rodando.
3. Disponibilize leituras por portas documentadas. Para valores com vários bytes, crie uma captura estável (snapshot) antes de ler os bytes: o contador pode mudar entre duas instruções in. Esse cuidado também vale para o relógio alto/baixo atual.
4. No `.asm`, leia os valores, converta-os para texto e imprima pela console até alcançar o critério de término. Use diferenças entre leitura inicial e final para medir um trecho; as próprias instruções de medição também fazem parte da execução.

8. Biblioteca C e roteiro final

Se você usar o compilador mcc em outros exercícios

O cabeçalho `compilador_c/inc/mancha.h` declara estas funções. Elas pertencem à biblioteca do projeto e não são instruções nativas da CPU. Nos exercícios exigidos em Assembly, implemente ou chame sub-rotinas Assembly com uma convenção compatível.

Função	Uso
<code>mancha_in(porta)</code>	Lê uma porta; nesta implementação, o dado retornado é de 8 bits.
<code>mancha_out(porta, valor)</code>	Escreve o byte do valor na porta.
<code>mancha_halt()</code>	Para o programa simulado.
<code>putchar(c) / getchar()</code>	Escreve um caractere / espera e lê um caractere.
<code>puts(texto)</code>	Escreve uma string e uma quebra de linha.
<code>print_int(v) / print_hex(v)</code>	Imprime decimal com sinal / quatro dígitos hexadecimais.

O mcc usa int de 16 bits e não oferece printf variádico. Para o T1 com números de 0 a 255, `print_int` atende essa faixa. A implementação atual merece revisão no extremo -32768, pois tenta negar esse valor em 16 bits; não trate a biblioteca como a libc completa do Linux.

Roteiro de estudo e entrega

Primeiro: execute `soma.asm`, `conta.asm` e `eco.asm`. Use 1 para um passo, C para continuar, P para pausar e F para sair. Em `eco`, envie entrada com E seguido do texto.

Depois: crie seu `.asm`, faça uma rotina simples imprimir um dígito e amplie para números maiores. Se exigido, implemente o dispositivo e conecte o programa à porta. Faça a leitura de dez valores antes de tentar ordenar ou medir desempenho.

Para entregar: inclua os `.c/.h` alterados, seus `.asm` escritos à mão, Makefile atualizado se houver novo módulo e um README com porta, faixa, política de semente/reinício, comandos e resultado esperado. Os binários e `.mob` podem ser recriados pelo professor. As sugestões de medição e ordenação não devem ser tratadas como obrigatórias sem confirmação no enunciado completo.

Fontes usadas para este guia

Trecho do T1 enviado na conversa; `simulador_completo/Makefile` e `README.md`; `src/dispositivos.h`, `dispositivos.c`, `cpu.h`, `cpu.c`, `main.c` e `instrucao.c`; `exemplos/conta.asm` e `eco.asm`; `compilador_c/inc/mancha.h`, `rt/runtime.asm`, `rt/biblioteca.c` e `README.md`. Caminhos `src/` e `exemplos/` desta lista são relativos a `simulador_completo/`.